

Chapter 1. Node Fundamentals

Node is an open source, cross-platform runtime environment in which developers can create backend services using the JavaScript language. It's built on top of V8, the JavaScript engine of the Chrome web browser, and it has dozens of built-in modules that are designed to be used asynchronously with an event-driven approach that's commonly known as the non-blocking model. Node developers can use events and handler functions to efficiently perform multiple operations in parallel, without having to deal with the complexity of multiple processes and threads.

There's a lot to unpack here, and that's what we will be doing in this first chapter. We'll start with an introduction to Node, how it works, and why it's popular. We'll learn the basics of the Node CLI, how to use modules and packages, and how to perform synchronous and asynchronous operations. We'll discuss the fundamentals of Node's event-driven, non-blocking model and learn how callbacks, promises, and events can be used to handle the result of an asynchronous operation.

NOTE

Throughout the book, I use the term *Node* instead of *Node.js* for brevity. The official name of the runtime environment is Node.js, but referring to it as just Node is common.

Introducing Node

Ryan Dahl started the Node project in 2009 after he was inspired by the performance of the V8 JavaScript engine in the Google Chrome web browser. V8 uses an *event-driven model*, which makes it efficient at han-

dling concurrent connections and requests. Ryan wanted to bring this same high-performance, event-driven architecture to server-side applications. The event-driven model is the first and most important concept you need to understand about Node (and the V8 engine as well). I'll explain it briefly in this chapter, and we'll expand on it in [Chapter 3](#).

TIP

I decided to give Node a spin and learn more about it after watching the presentation Ryan Dahl gave to introduce it. I think you'll benefit by starting there as well. Search YouTube for "[Ryan Dahl introduction to Node](#)". Node has changed significantly since then, so don't focus on the examples but rather the concepts and explanations.

In its core, Node enables developers to use the JavaScript language on any machine without needing a web browser. Node is usually defined as "JavaScript on backend servers." Before Node, that was not a common or easy thing. JavaScript was mainly a frontend thing.

However, this definition isn't completely accurate. Node offers a lot more than the ability to execute JavaScript on servers. In fact, the actual execution of JavaScript is done by the V8 JavaScript engine, not Node. Node is just an interface to V8 when it comes to executing JavaScript code.

V8 is Google's open source JavaScript engine that can compile and execute JavaScript code. It's used in Node as well as in Chrome and a few other browsers. It's also used in Deno, the new JavaScript runtime that was created by Ryan Dahl in 2018.

NOTE

There are other JavaScript engines, like SpiderMonkey, which is used by Firefox, and JavaScriptCore, which is used by the Safari web browser and in Bun, an all-in-one JavaScript runtime, package manager, and bundler.

Node is better defined as a server runtime environment that wraps V8 and provides modules to help developers build and run efficient software applications with JavaScript.

The key word in this definition is *efficient*. Node adopts and expands on the same event-driven model that V8 has. Most of Node's built-in modules are event-driven and can be used asynchronously without blocking the main *thread* of execution that your code runs in.

A thread is basically a small process within a larger one. A process can create multiple threads of execution that are each associated with a CPU core. Threads can share memory and resources within the larger process.

In multithreaded programming, slow operations are executed in separate threads. In Node, you get a single main thread for your code, and all the slow operations are executed outside of that main thread, asynchronously.

You need to read the content of an external file? You can do that asynchronously without blocking the single main thread. You need to start a web server? Work with network sockets? Parse, compress, or encrypt data? Every low-level slow operation has an asynchronous API for you to use without blocking your other operations.

You don't need to deal with multiple threads to do things in parallel in Node. You don't waste resources on manual threads being idle waiting on slow operations. You code in one thread and use asynchronous APIs, and Node takes care of executing the asynchronous operations efficiently outside of your main thread.

Any code that needs to be executed after a slow operation can be managed with *events* and *event handlers*. An event is a signal that something has happened and a certain action needs to be performed. The action can be defined in an event handler function that gets associated with the event. Every time the event is signaled, its handler function will be executed. That's basically the gist of what *event-driven* means.

We'll expand on these important concepts once we learn the basics of running Node code and using its modules and packages.

The JavaScript Language

After considering programming languages like Python, Lua, and Haskell, Ryan Dahl picked the JavaScript language for Node because it was a good fit. It's simple, flexible, and popular, but more importantly, JavaScript functions are first-class citizens that we can treat like any other objects (numbers or strings). We can store them in variables, pass them to other functions via arguments, and even return them from other functions, all while preserving their state. Node leveraged that to implement its handling of asynchronous operations.

NOTE

Despite JavaScript's historical problems, I believe it's a decent language today that can be made even better by using TypeScript (which we will discuss in [Chapter 10](#)).

Besides simplifying the implementation of asynchronous operations, the fact that JavaScript is the programming language of browsers gave Node the advantage of having a single language across the full stack. There are some subtle but great benefits to that:

- One language means less syntax to keep in your head, fewer APIs and tools to work with, and fewer mistakes overall.
- One language means better integrations between your frontend code and your backend code. You can actually share code between these two sides. For example, you can build a frontend application with a JavaScript framework like React, then use Node to render the same components of that frontend application on the server and generate initial HTML views for the frontend application. This is known as server-side rendering (SSR), and it's something that many Node frontend frameworks offer out of the box.
- One language means teams can share responsibilities among different projects. Projects don't need a dedicated team for the frontend and a

different team for the backend. You would also eliminate some dependencies between teams. A full stack project can be assigned to a single team, *The JavaScript People*; they can develop APIs, they can develop web and network servers, they can develop interactive websites, and they can even develop mobile and desktop applications. Hiring JavaScript developers who can contribute to both frontend and backend applications is attractive to employers.

Executing Node Code

If you have Node installed on your computer, you should have the commands `node` and `npm` available in a terminal. If you have these commands, make sure the Node version is a recent one (20.x or higher). You can verify that by opening a terminal and running the command `node -v`.

If you don't have the `node` command, you'll need to download and install Node from the [Node website](#). The installation process is straightforward and should only take a few minutes.

For macOS users, Node can also be installed using the Homebrew package manager with the command:

```
$ brew install node
```

NOTE

Throughout this book, I use the `$` sign to indicate a command line to be executed in a terminal. The `$` sign is not part of the command. It's a common prompt character in terminals.

Another option to install Node is using Node Version Manager (NVM). NVM allows you to run multiple versions of Node and switch between them easily. You might need to run a certain project with an older version of Node, and use the latest Node version with another project. NVM

works on Mac and Linux, and there's a Windows option as well, called *nvm-windows*.

NODE ON WINDOWS

All the examples I will be using in this book are designed for a macOS environment and should also work for a Linux-based OS. On Windows, you need to switch the commands I use with their Windows alternatives.

I don't recommend using Node on Windows natively unless it's your only option. If you have a modern Windows machine, one option that might work a lot better for you is to install the Windows subsystem for Linux. This option will give you the best of both worlds. You'll have your Windows OS running Linux without needing to reboot. You can even edit your code in a Windows editor and execute it in Linux!

If you're using NVM, install the latest version of Node with the command:

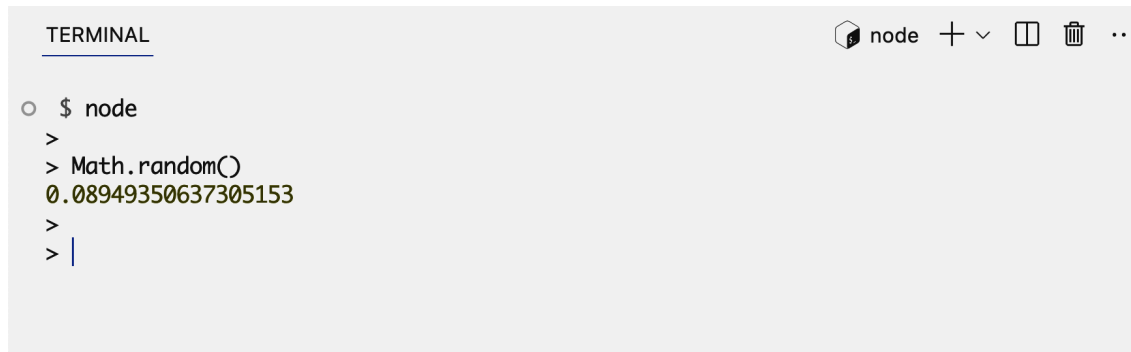
```
$ nvm install node
```

TIP

Major Node versions are released frequently. When a new version is released, it enters a *Current* release status for six months to give library authors time to make their libraries compatible with the new version. After six months, odd-numbered releases (19, 21, etc.) become unsupported, and even-numbered releases (18, 20, etc.) move to *Active LTS* (long-term support) status. LTS releases typically guarantee that critical bugs will be fixed for a total of 30 months. Production applications should use only Active LTS releases.

Once you have the `node` command ready, open a terminal and issue the command on its own without any arguments. This will start a Node *REPL* session. REPL stands for Read-Eval-Print-Loop. It's a convenient way to quickly test simple JavaScript and Node code. You can type any JavaScript

code in a REPL session. For example, try a `Math.random()` line, as shown in [Figure 1-1](#).

A screenshot of a terminal window titled "TERMINAL". The window has a dark background and a light-colored text area. The text area shows a Node.js REPL session. The prompt is "\$ node". The user enters ">". The prompt changes to ">". The user enters "> Math.random()". The terminal outputs "0.08949350637305153". The prompt changes to ">". The user enters ">". The prompt changes to "> |".

```
TERMINAL
○ $ node
>
> Math.random()
0.08949350637305153
>
> |
```

Figure 1-1. Node's REPL mode

Node will read the line, evaluate it, print the result, and loop over these three things for everything you type until you exit the session (which you can do with Ctrl + D). Note how the *Print* step happened automatically. We didn't need to add any instructions to print the result. Node will just print the result of each line you type. This is not the case when you execute code in a Node script. Let's do that next.

NOTE

We'll learn more about Node's REPL mode in [Chapter 2](#).

Create a new directory for the exercises of this book, and then `cd` into it:

```
$ mkdir efficient-node
$ cd efficient-node
```

Open up your code editor and create a file named `test.js`. Put the same `Math.random()` line into it:

```
Math.random();
```

Now to execute that script, in the terminal type the following command:

```
$ node test.js
```

You'll notice that the command will basically do nothing. That's because we did not *output* anything in that script. To output something, you can use the global `console` object, which is similar to the one available in browsers. Here is an example:

```
console.log(  
  Math.random()  
);
```

Executing *test.js* now will output a random number, as shown in [Figure 1-2](#). In this simple example, we're using both JavaScript (`Math` object) and an object from the Node API (`console`). The `console.log` method writes the value of its arguments to the default standard output stream (*stdout*) of the running process.

NOTE

The `console` object is one of many top-level *global scope* objects that we can access in Node without needing to declare any dependencies. Similar to how the global `window` object in browsers can be accessed with the `globalThis` property, in Node, the `globalThis` property is the `global` object and the `console` object is part of it. All properties of `globalThis` can be accessed directly: for example, `console.log` instead of `globalThis.console.log` (which also works).

A screenshot of a web-based code editor. The top bar shows a search icon and the text 'efficient-node'. Below it, a tab is labeled 'JS test.js'. The editor area contains a JavaScript file named 'test.js' with three lines of code:

```
1 console.log(  
2   | Math.random()  
3 );
```

 Below the code editor is a terminal window titled 'TERMINAL'. It shows the command `$ node test.js` being executed, followed by the output `0.674001259615383`. The terminal also shows the prompt `$` and a cursor.

Figure 1-2. Executing a Node script

Using Built-In Modules

You can create a simple web server in Node using its built-in `node:http` module.

Create a `server.js` file and write the following code in it:

```
// Basic Web Server Example  
  
const { createServer } = require('node:http');  
  
const server = createServer((req, res) => {  
  res.writeHead(200, { 'Content-Type': 'text/plain' });  
  res.end('Hello World');  
});  
  
server.listen(3000, '127.0.0.1', () => {  
  console.log('Server is running...');  
});
```

This is Node’s version of a “Hello World” example. You don’t need to install anything to run this script. This is all Node’s built-in power.

When you execute this script, Node creates a web server and runs it on `http://127.0.0.1:3000`, as shown in [Figure 1-3](#).

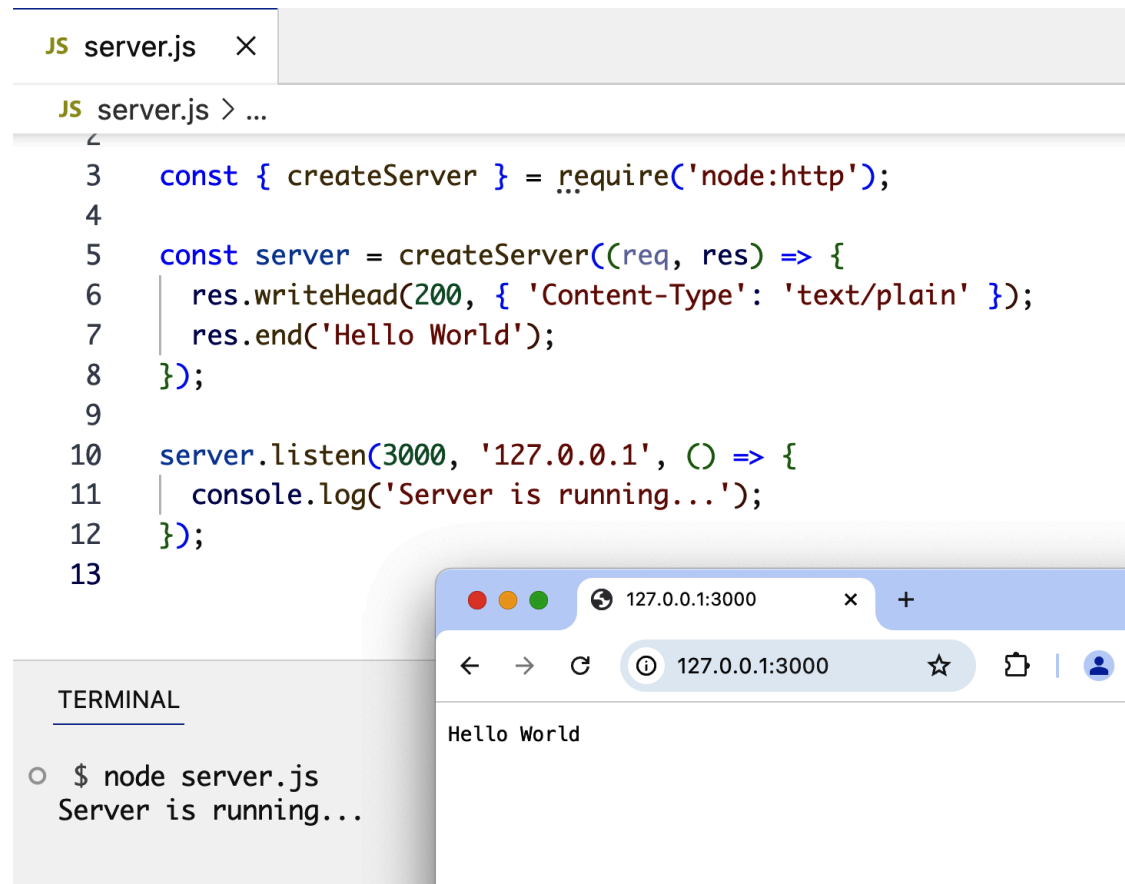


Figure 1-3. A basic web server

TIP

Note how in this example, the Node process continues to run indefinitely (unless it encounters any unexpected errors). This is because it has work to do in the background. It needs to wait for users to request data and send them a response when they do.

While this web server example is a basic one, it has a few important concepts to understand. Let's go over it in detail.

The `require` function is part of Node's original dependency management method. It allows a module (like `server.js`) to use the features of another module (like `node:http`). By requiring the `node:http` module, the `server.js` module now *depends* on it. It cannot run without it.

Another way for one module to use the features of another module is with an ES modules `import` statement. ES modules are the modern ECMAScript standard for working with modules in JavaScript. We'll be mostly using them in this book as they are the preferred module system to use in Node today. However, it's good to learn the original module management system (which is known as CommonJS) as many Node projects and libraries are built using that legacy system, and it's very likely that you will have to deal with them even if you're starting a project from scratch.

There are many libraries that you can use to create a web server, but `node:http` is part of Node itself. You don't need to install anything to use it, but you do need to require (or import) it.

TIP

In a Node's REPL session, built-in modules (like `node:http`) are available globally without needing to require them. This is not the case with executable scripts. You can't use modules (including built-in ones) without declaring them first.

You don't need to load everything in a module when you require it. You can pick and choose. This example loads only the `createServer` function, which is one of many functions and other objects that are available in `node:http`.

We invoke `createServer` to create a server object. Its argument is another function that is known as the `RequestListener`. Don't worry about the syntax in this example; focus on the concepts.

A listener function in Node is associated with a certain event, and it gets executed when its event is triggered. In this example, Node will execute the `RequestListener` function every time there is an incoming connection request to the web server. That's the event associated with this listener function.

The listener function receives two arguments:

The request object (named `req` in this example)

You can use this object to read data about incoming requests. For example, what URL is being requested, or what is the IP address of the client that's making a request.

The response object (named `res` in this example)

You can use this object to write things back to the requester. It's exactly what this simple web server is doing. It's setting the response status code to `200` to indicate a successful response, and the `Content-Type` header to `text/plain`. Then it's writing back the `Hello World` text using the `end` method on the `res` object.

The `createServer` function only creates the server object. It does not activate it. To activate this web server, you need to invoke the `listen` method on the created server.

The `listen` method accepts many arguments, like what OS port and host to use for this server. The last argument for it is a function that will be invoked once the server is successfully running on the specified port. This example prints a console message to indicate that the server is running successfully at that point.

Both functions received by the `createServer` and `listen` methods are examples of handler functions that are associated with events related to an asynchronous operation. We'll learn how these events and their handler functions are managed in [Chapter 3](#).

TIP

Note how I use a `node:` prefix when working with the built-in modules in Node. This is a helpful practice to distinguish them from external modules and identify their built-in nature. Since it's required for a few modules (like `node:test`), it's good to be consistent and use it for all modules.

To stop the web server, press `Ctrl + C` in the terminal where it's running.

Using Packages

Node's package manager is npm. It's a simple CLI that lets us install and manage external *packages* in a Node project. An npm package can be a single module or a collection of modules grouped together and exposed with an API. We'll talk more about npm and its commands and packages in [Chapter 5](#). Here, let's just look at a simple example of how to install and use an npm package.

Let's use the popular `lodash` package, which is a JavaScript utility library with many useful methods you can run on numbers, strings, arrays, objects, and more.

First, you need to download the package. You can do that using the `npm install` command:

```
$ npm install lodash
```

This command will download the `lodash` package from the npm registry, and then place it under a `node_modules` folder (which it will create if it's not there already). You can verify with an `ls` command:

```
$ ls node_modules
```

You should have a folder named `lodash` in there.

Now in your Node code, you can `require` the `lodash` module to use it. For example, `lodash` has a `random` method that can generate a random number between any two numbers you pass to it. Here's an example of how to use it:

```
const _ = require('lodash');

console.log(
  _.random(1, 99)
);
```

When you run this script, you'll get a random number between 1 and 99, as shown in [Figure 1-4](#).



The image shows a code editor window with a tab labeled 'JS index.js'. The code in the editor is as follows:

```
JS index.js > ...
1  const _ = require('lodash');
2
3  console.log(
4    |  _.random(1, 99)
5  );
6
```

Below the code editor is a terminal window titled 'TERMINAL'. It shows the command `$ node index.js` being executed, which results in the output `56`. The terminal also shows the prompt `$` and a cursor.

Figure 1-4. Using an npm package

TIP

The underscore (`_`) is a common name to use for `lodash`, but you can use any name.

Since we called the `require` method with a non-built-in module, `lodash`, Node will look for it under the `node_modules` folder, and thanks to npm, it'll find it there.

In a team Node project, when you make the project depend on an external package like this, you need to let other developers know of that dependency. You can do so in Node using a `package.json` file at the root of the project.

When you `npm install` a module, the `npm` command will also list the module and its current version in `package.json`, under a `dependencies` section. Look at the `package.json` file that was autocreated when you installed the `lodash` package and see how the `lodash` dependency was documented (see [Figure 1-5](#)).

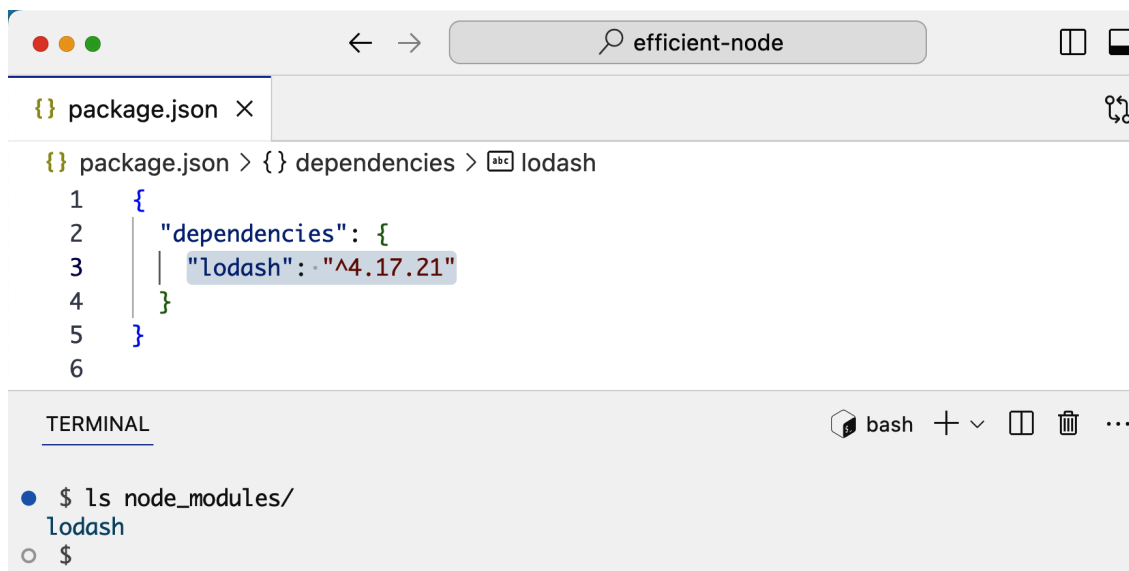


Figure 1-5. The package.json file

The *package.json* file can also contain information about the project, including the project's name, version, description, and more. It can also be used to specify scripts that can be run from the command line to perform various tasks, like building or testing the project.

Here's an example of a typical *package.json* file:

```
{
  "name": "efficient-node",
  "version": "1.0.0",
  "description": "A guide to learning Node.js",
  "license": "MIT",
  "scripts": {
    "start": "node index.js"
  },
  "dependencies": {
    "lodash": "^4.17.21"
  }
}
```

You can interactively create a *package.json* file for a new Node project using the `npm init` command:

```
$ npm init
```

This command will ask a few questions, and you can interactively supply your answers (or press Enter to keep the defaults, which often are good because the command tries to detect what it can about the project).

Try to `npm install` new packages (for example, `chalk`) and see how it gets listed as a dependency in your `package.json` file. Then `npm uninstall` the package and see how it gets removed from `package.json`.

Your `package.json` file will eventually list many dependencies. When other developers pull your code, they can run the command `npm install` without any arguments, and npm will read all the dependencies from `package.json` and install them under the `node_modules` folder.

Some packages are needed only in a development environment, not in a production environment. The ESLint package is an example of that. You can instruct the `npm install` command to list a package as a development-only dependency by adding the `--save-dev` argument (or `-D` for short):

```
$ npm install -D eslint
```

This will install the `eslint` package in the `node_modules` folder, and document it as a development dependency under a `devDependencies` section in `package.json`. This is where you should place things like your testing framework, your formatting tools, or anything else that you use only while developing your project.

TIP

In addition to `dependencies` and `devDependencies`, a `package.json` file can also specify `optionalDependencies` for packages that are optional, and `peerDependencies` for packages that need to work alongside other packages but do not directly depend on them. Peer dependencies are needed only by package authors.

If you take a look at what's under `node_modules` after you install `eslint`, you'll notice that there are dozens of new packages there (see [Figure 1-6](#)).

The `eslint` package depends on all these other packages. Be aware of these indirect dependencies in the future. By depending on one package, a project is indirectly depending on all of that package's dependencies, and the dependencies of all the subdependencies, and so on. With every package you install, you add a tree of dependencies.

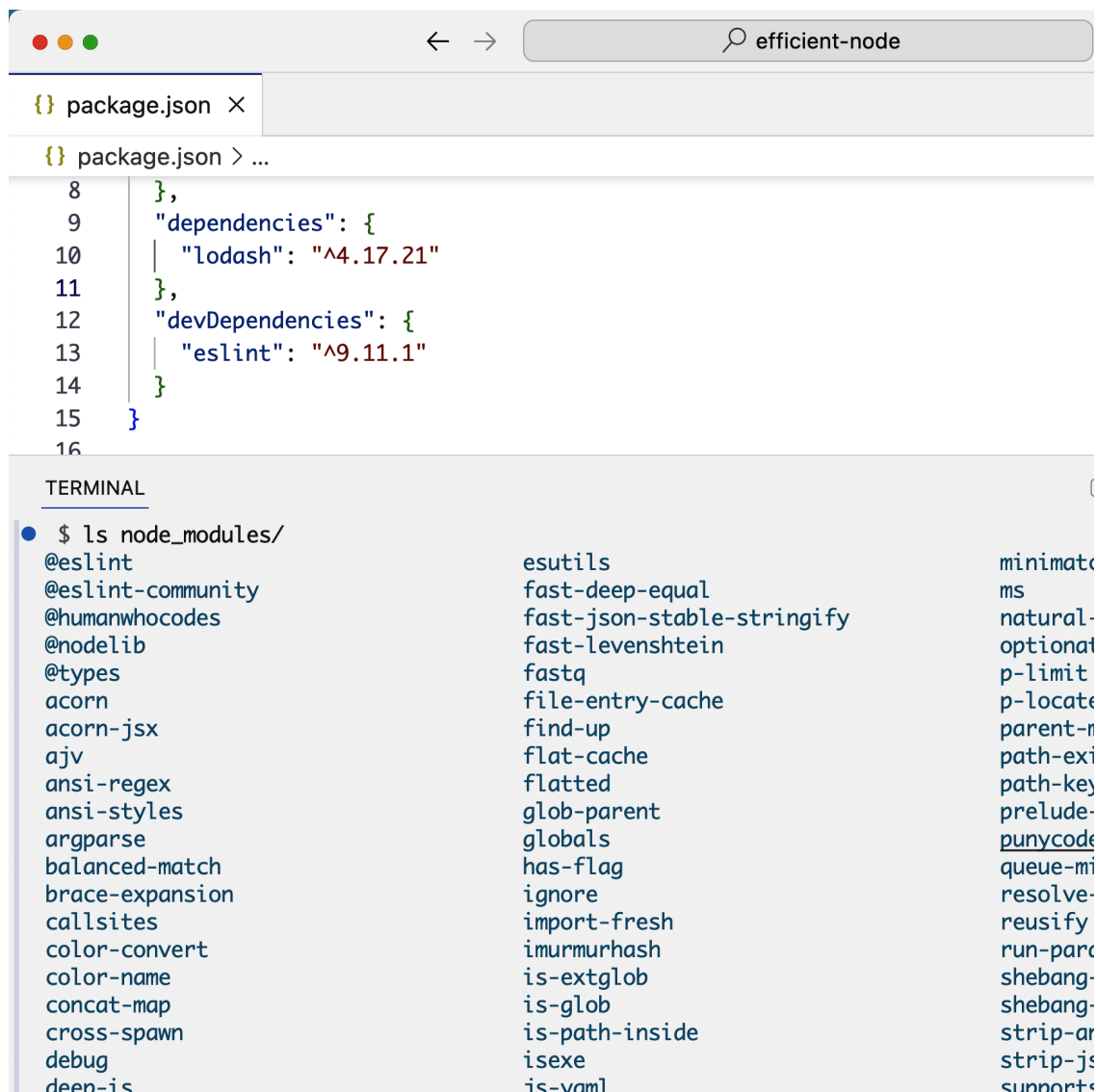


Figure 1-6. npm packages and their indirect dependencies

Some packages can also be installed (and configured) directly with the `init` command. ESLint is an example of a package that needs a configuration file before you can use it. The following command will install ESLint and create a configuration file for it (after asking you a few questions about your project):

```
$ npm init @eslint/config@latest
```

TIP

In a production machine, development dependencies are usually ignored. The `npm install` command has a `--production` flag to make it ignore them. You can also use the `NODE_ENV` environment variable and set it to `production` before you run the `npm install` command. We'll learn more about Node environments and variables in [Chapter 2](#).

ES Modules

Node has two different module loaders. The default one is the CommonJS module loader, and we saw an example of it that uses the `require` function.

The other one is the JavaScript-native ES module loader that is supported in modern browsers as well. In ES modules, we use `import` statements to declare a module dependency and `export` statements to make one module's features available for other modules to use.

One important difference between these two module systems is that CommonJS modules get loaded dynamically at runtime, while ES module dependencies are determined at compile time, allowing them to be statically analyzed and optimized. For example, with ES modules we can easily find what code is not being used and exclude it from the compiled version of the application.

TIP

While the two module types can be used together, you need to be careful about mixing them. CommonJS modules are synchronous while ES modules are asynchronous.

To see ES modules in action, let's expand on the basic web server example code and split it into two files: one to create the web server, and one to

run it.

The simplest way to use ES modules in Node is to save files with a *.mjs* file extension instead of a *.js* extension. This is because by default, Node assumes that all *.js* extensions are using the CommonJS module system. This is configurable though.

To make Node treat all *.js* files as ES modules, you can add a `type` key in *package.json* and give it the value of `module` (the default value for it is `commonjs`). You can do that manually or with this command:

```
$ npm pkg set type=module
```

With that, you can now use ES modules with the *.js* extension.

NOTE

Regardless of what default module type you use, Node will always assume a *.mjs* file is an ES module file and a *.cjs* file is a CommonJS module file. You can import a *.cjs* file into an ES module, and you can import a *.mjs* file into a CommonJS module.

Let's modify the basic web server example to use ES modules. In the *server.js* file, write the following code:

```
import { createServer } from 'node:http';

export const server = createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain' });
  res.end('Hello World');
});
```

Note the use of `import` / `export` statements. This is the syntax for ES modules. You use `import` to declare a module dependency and `export` to define what other modules can use when they depend on your module.

In this example, the *server.js* module exports the `server` object, enabling other modules to import it and depend on it.

To use the *server.js* exported objects in other modules, we use another import statement. In an *index.js* file, write the following code:

```
import { server } from './server.js';

server.listen(3000, () => {
  console.log('Server is running...');
});
```

The `./` part in the import line signals to Node that this import is a relative one. Node expects to find the *server.js* file in the same folder as *index.js*. You can also use a `../` to make Node look for the module up one level, or `../../` for two levels, and so on. Without `./` or `../`, Node assumes that the module you're trying to import is either a built-in module or a module that exists under the *node_modules* folder.

With this code, the *index.js* module depends on the *server.js* module and uses its exported `server` object to run the server on port 3000.

Execute *index.js* to start the web server and test it, as illustrated in [Figure 1-7](#).

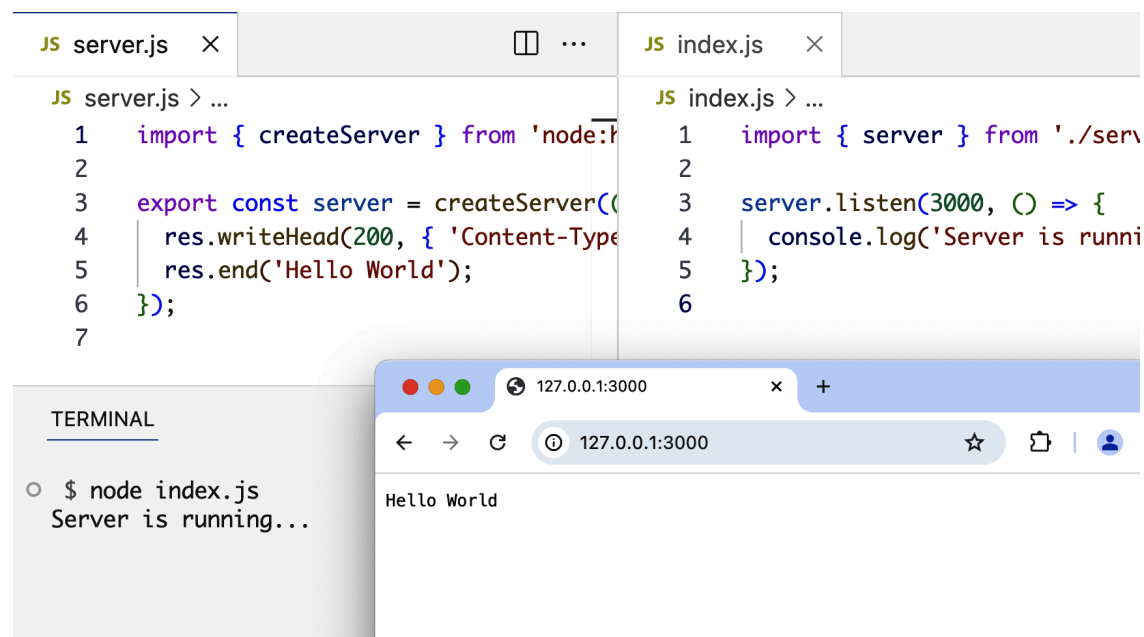


Figure 1-7. An ES modules web server

The `export` object syntax is known as *named exports* and it's great when you need to export multiple things in a module. You can use the `export` keyword to prefix any object, including functions, classes, and destructured variables:

```
export function functionName() { ... }
export class ClassName { ... }
export const [name1, name2]
```

You can also use one `export` keyword, usually at the end of a module, to export all named objects together:

```
export {
  functionName,
  ClassName,
  name1,
  name2,
  // ...
};
```

You can import these named exports individually, or use the `* as` syntax to import all of them:

```
// To import named exports individually:
import { functionName, name1 } from './module'

// To import all named exports:
import * as serviceName from './module'

// Then access named exports as:
// serviceName.functionName
// serviceName.name1
```

In addition to the named export syntax, ES modules also have a *default export* syntax:

```
// To export the server object
// as the default export in server.js:
```

```
export default server;

// To import it, you need to
// specify a name:
import myServer from './server.js';

// Or:
import { default as myServer } from './server.js';
```

Note that to import a default export, you have to name it, while with named exports you don't (although you can if you need to). Named exports are better for consistency, discoverability, and maintainability.

TIP

Since a module might not have a default export, you can always use the `* as` syntax since it works with or without a default export.

The `export` / `import` keywords support other syntax like renaming and exporting from other modules, but in general my recommendation is to avoid using default exports; always use named exports, and keep them simple and consistent. For example, I always specify my named exports as the last line of the module with a single `export { ... }` statement.

Real-life analogies can sometimes help us understand coding concepts.

One of my favorite analogies about coding in general is how it can be compared to writing cooking recipes. The recipe in this analogy is the program, and the cook is the computer.

In some recipes, you can use premade items like a cake mix or a special sauce. You would also need to use tools like a pan or a strainer. When compared to coding, you can think of these premade items and tools as the packages of code written by others, which you can just download and use.

Extending on this analogy, you can think of an npm registry as the store where you get your premade items and tools for your coding recipes.

But what exactly is Node's place in this analogy?

I like to think of it as the kitchen! It allows you to execute lines in your coding recipes by using built-in tools, like the stove and the sink.

Now imagine trying to cook without these built-in tools in your kitchen. That would make the task a lot harder, wouldn't it?

Asynchronous Operations

While static import declarations are preferable for loading initial dependencies, there are many cases where you will need to import modules dynamically. For example:

- When a module is slowing the loading of your code and it's not needed until a later time
- When a module does not exist at load time
- When you need to dynamically construct the name of the module to import
- When you need to import a module conditionally

For these cases, we can use the `import()` function. It's similar to the `require()` function, but it's asynchronous.

Let's think about an example where we need to read the content of a file before starting the basic web server. We can simulate the file reading delay using a simple `setTimeout` function.

NOTE

Timer functions can be used to delay the execution of a function or make it repeat regularly. If you're not familiar with them, the next sidebar explains them with examples.

Since we don't need the `server.js` module until a later point in time, we can import it with the `import()` function when we are ready for it:

```
setTimeout(async () => {  
  const { server } = await import('./server.js');  
  
  server.listen(3000, () => {  
    console.log('Server is running...');  
  });  
}, 5_000);
```

If you execute this code, the Node process will wait five seconds. It will then dynamically import the `server.js` module and use it to start the server.

This example introduces the important `Promise` object concept and how to consume it with the `async/await` syntax. This is the modern JavaScript syntax to handle asynchronous operations. We'll learn more about promises and the `async/await` syntax in the next section.

TIP

Dynamic import expressions can be used in CommonJS modules to load ES modules.

TIMER FUNCTIONS

Timer functions in Node like `setTimeout` and `setInterval` behave similarly to how they do in browser environments. A timer function receives a function as an argument:

```
const printGreeting = () => console.log('Hello');

setTimeout(printGreeting, 4_000);
```

The `printGreeting` function will be invoked once after four seconds. To invoke it multiple times, you can use the `setInterval` function. Replacing `setTimeout` with `setInterval` in this example will make Node print the “Hello” message every four seconds, forever.

Timer functions can be canceled once they are defined. When you call a timer function, you get back a unique timer ID. You can use that timer ID to cancel the scheduled timer. We can use `clearTimeout(timerId)` to stop timers started by `setTimeout`, and `clearInterval(timerId)` to stop timers started by `setInterval`:

```
const timerId = setTimeout(
  () => console.log('Hello'),
  0,
);

clearTimeout(timerId);
```

In this example, even though we started a timer to print a message after zero milliseconds, the message will *not* be printed at all because we canceled the asynchronous timer operation after it was defined.

The Non-Blocking Model

When you need to perform a slow operation in your code (like reading a file from the filesystem), you’ll need a way to handle the output of that

slow operation.

Let's simulate a slow operation function with a long-running `for` loop:

```
function slowOperation() {  
  for (let i = 0; i <= 1e9; i++) {  
    // ...  
  }  
  return { success: true };  
}
```

The `slowOperation` function might return data successfully or throw an error (like a timeout error). Here's a simple example function to handle its output:

```
function handlerFunction(output) {  
  if (!output.success) {  
    // Something went wrong  
  }  
  // Do something with output  
}
```

Here's how we would use the `slowOperation` function to pass its output to its `handlerFunction`:

```
const output = slowOperation();  
handlerFunction(output)  
  
console.log('Hello');  
// Other operations
```

The problem with this is that the slow operation will block the execution of all other operations that follow it. The `console.log` operation will wait until both `slowOperation` and `handlerFunction` are done executing.

Since JavaScript functions can be passed as arguments, we can design `slowOperation` to invoke its handler function once it's done, using the

following pattern:

```
function slowOperation(callbackFunction) {
  for (let i = 0; i <= 1e9; i++) {
    // ...
  }
  callbackFunction({ success: true });
}

slowOperation(
  (output) => handlerFunction(output)
);

// Other operations
```

Now, we can make the `slowOperation` run in a different thread so that the other operations in the main thread are not blocked. This is known as the *callback pattern* and it's the original implementation of handling asynchronous operations in Node. A callback function gets called at a later point in time once the slow operation is done.

The `setTimeout` function is the simplest example of an asynchronous function that follows the callback pattern:

```
setTimeout(
  function callback() {
    console.log('World');
  },
  2_000, // delay is in ms
);

console.log('Hello');
```

The `setTimeout` function itself is not part of JavaScript. It's implemented in the runtime environment like Node (or the browser). What gets executed by the JavaScript engine is its callback function.

TIP

You can think of the callback pattern as a simple method for performing an asynchronous operation with a handler function and a built-in event. For `setTimeout`, the built-in event is a time delay. The handler function is what we pass to `setTimeout` as a callback function.

The actual timer operation is handled in a separate thread so that it does not block the main thread. That's why the output of this will be as follows:

```
Hello
World
```

The operation that follows the timer operation was executed first. Then, once the built-in event was triggered (two seconds passed by), the callback function that internally gets associated with this timer event was executed by V8.

This would also be the output when the timer delay is zero. All asynchronous operations, no matter how fast they are, get removed from the main thread immediately, processed internally in Node, and return to the main thread (via their callbacks) once all the other synchronous operations are done.

Zero-milliseconds delayed code is a way to *schedule* code to be invoked when all the synchronous code defined after it is done executing. This is a good simple example of the non-blocking nature of Node. If we define the long-running `for` loop in the callback of a `setTimeout` function with a delay of zero, we are basically scheduling that loop to execute after all the synchronous operations that come after it are done:

```
setTimeout(  
  () => {  
    for (let i = 0; i <= 1e9; i++) {  
      // ...  
    }  
  },  
  0,
```

```
);  
  
console.log('Hello');
```

The “Hello” message will print first here, then the long-running loop will be executed.

TIP

A general observation about timer functions is that their delays are not guaranteed to be exact, but rather a minimum amount. Delaying a function by 10 milliseconds means that the execution of that function will happen after a minimum of 10 milliseconds, but possibly longer depending on the code that comes after it.

A few years after the success of Node and its use of the callback pattern, `Promise` objects were introduced to the JavaScript language. A `Promise` object represents a value that might be available in the future. This enables us to natively wrap an asynchronous operation as a `Promise` object to which handler functions can be attached and executed later once the promise value is resolved.

Here’s the main pattern for `Promise` objects applied to our simple `slowOperation` and `handlerFunction` example:

```
const outputPromise = slowOperation();  
outputPromise.then(  
  (output) => handlerFunction(output)  
);  
  
// Other operations
```

The `node:timers` module has a promise-based `setTimeout` function that can be used with this pattern:

```
import { setTimeout } from 'node:timers/promises';  
  
setTimeout(2_000).then(  
  () => {  
    // ...  
  })
```

```
function callback() {  
  console.log('World');  
}  
);  
  
console.log('Hello');
```

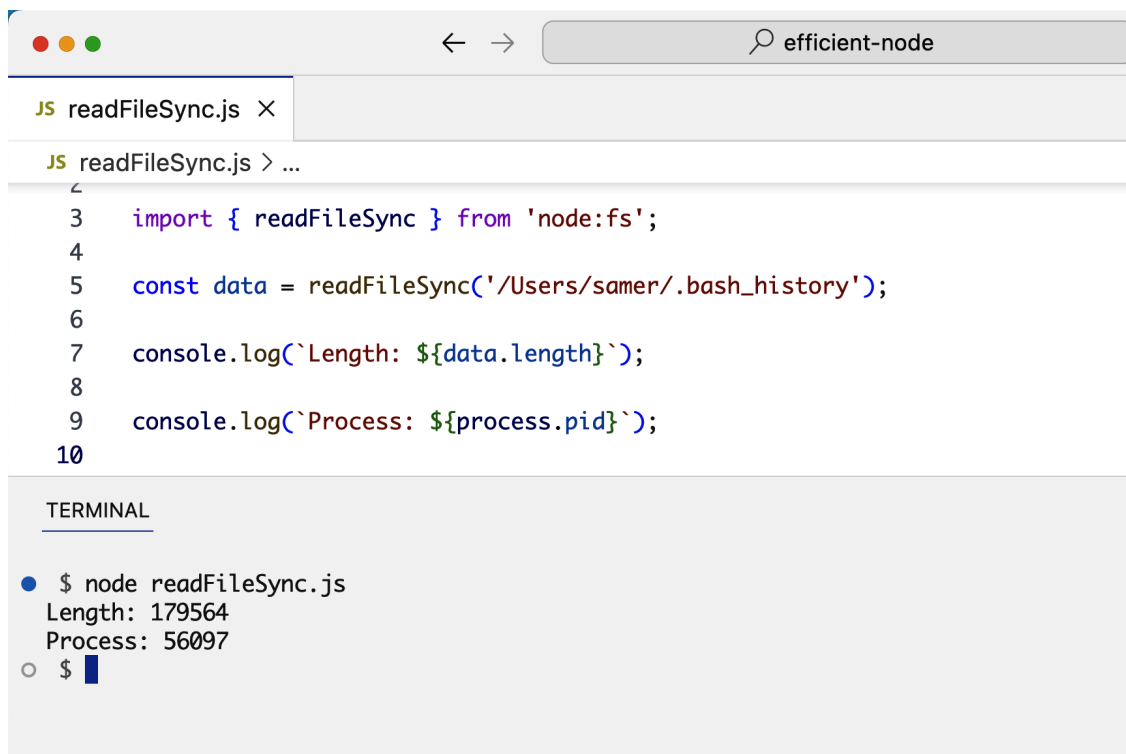
This will be equivalent to the callback-based example. The “Hello” message will be printed immediately, then after a two-second delay, the “World” message will be printed.

The callback and promise patterns can both be used in Node today to use the asynchronous APIs of its built-in modules. Let’s look at an example from the `node:fs` module, which we can use to read the content of a file from the filesystem.

Here’s the simplest way to do that:

```
// Reading a file synchronously  
  
import { readFileSync } from 'node:fs';  
  
const data = readFileSync('/Users/samer/.bash_history');  
  
console.log(`Length: ${data.length}`);  
  
console.log(`Process: ${process.pid}`);
```

[Figure 1-8](#) shows the output of running this code.



```
JS readFileSync.js x
JS readFileSync.js > ...
2
3 import { readFileSync } from 'node:fs';
4
5 const data = readFileSync('/Users/samer/.bash_history');
6
7 console.log(`Length: ${data.length}`);
8
9 console.log(`Process: ${process.pid}`);
10
```

TERMINAL

- \$ node readFileSync.js
Length: 179564
Process: 56097
- \$

Figure 1-8. Reading a file synchronously

Reading a file is an I/O operation, and it's done synchronously in this example. This means it'll block the main thread, and any code that's written after it will have to wait until it's done. Note how the `console.log` statement for the process PID had to wait until after the reading operation was done.

This is bad, especially if you're trying to read a big file. If this code was part of a web server, all incoming requests to that server would have to wait until the main thread is not blocked anymore. We'll test an example of that in [Chapter 3](#).

NOTE

An *I/O operation* refers to any communication between a computer program process and its outside world. It typically involves the transfer of data to/from storage devices (like hard drives and memory), peripheral devices (like a keyboard, mouse, or printer), or over a network. I/O operations can be slow, and that's why they are usually run in different processes to not block the main thread of execution.

Performing I/O operations synchronously like this might be OK in a few cases. For example, if you need to read a file one time before you start a web server, or right before you stop it, you can do that synchronously. In most other cases, you want to avoid using any synchronous operations and use only non-blocking ones. Here's how you can read the content of a file asynchronously and avoid blocking the main thread:

```
// Reading a file asynchronously

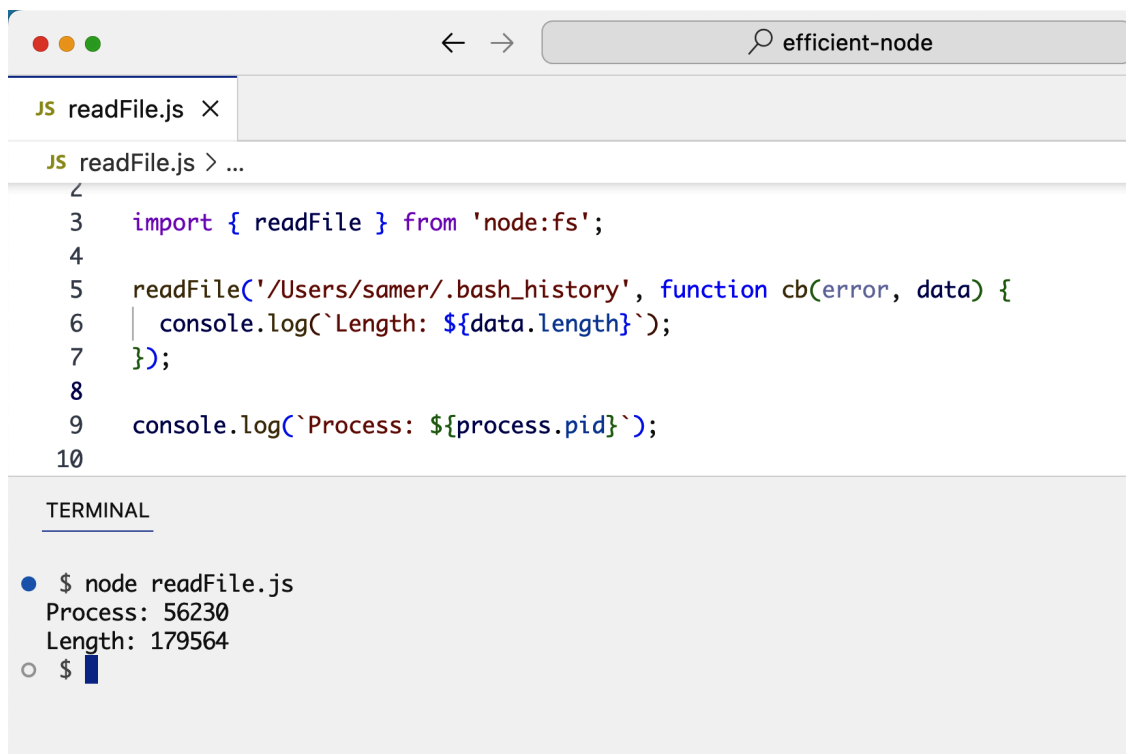
import { readFile } from 'node:fs';

readFile('/Users/samer/.bash_history', function cb(error, data) {
  console.log(`Length: ${data.length}`);
});

console.log(`Process: ${process.pid}`);
```

[Figure 1-9](#) shows the output of running this code. Note how the `console.log` statement for the process PID was executed before the `console.log` statement for the file data length. The file reading operation did not block the main thread.

This is because the `readFile` method is an asynchronous one. Node does not execute it in the main thread at all. It takes it elsewhere and schedules the execution of its associated callback function right after the reading operation is done.



The screenshot shows a code editor window with a tab for 'JS readFile.js'. The code in the editor is as follows:

```
2
3 import { readFile } from 'node:fs';
4
5 readFile('/Users/samer/.bash_history', function cb(error, data) {
6   console.log(`Length: ${data.length}`);
7 });
8
9 console.log(`Process: ${process.pid}`);
10
```

Below the code editor is a terminal window titled 'TERMINAL'. It shows the command '\$ node readFile.js' being executed, followed by the output 'Process: 56230' and 'Length: 179564'. The prompt '\$' is visible at the bottom.

Figure 1-9. Reading a file asynchronously

In this simple example, the callback function is associated with the `readFile` method itself, but internally, that translates to it being associated with an implicit event that gets triggered when the file data is ready.

You'll soon see examples of functions associated with explicit events, either built-in events or user-defined events.

Here's how this example can be converted into using a `Promise` object instead of a callback function:

```
// Reading a file asynchronously with promises

import { readFile } from 'node:fs/promises';

async function logFileLength() {
  const data = await readFile('/Users/samer/.bash_history');
  console.log(`Length: ${data.length}`);
}

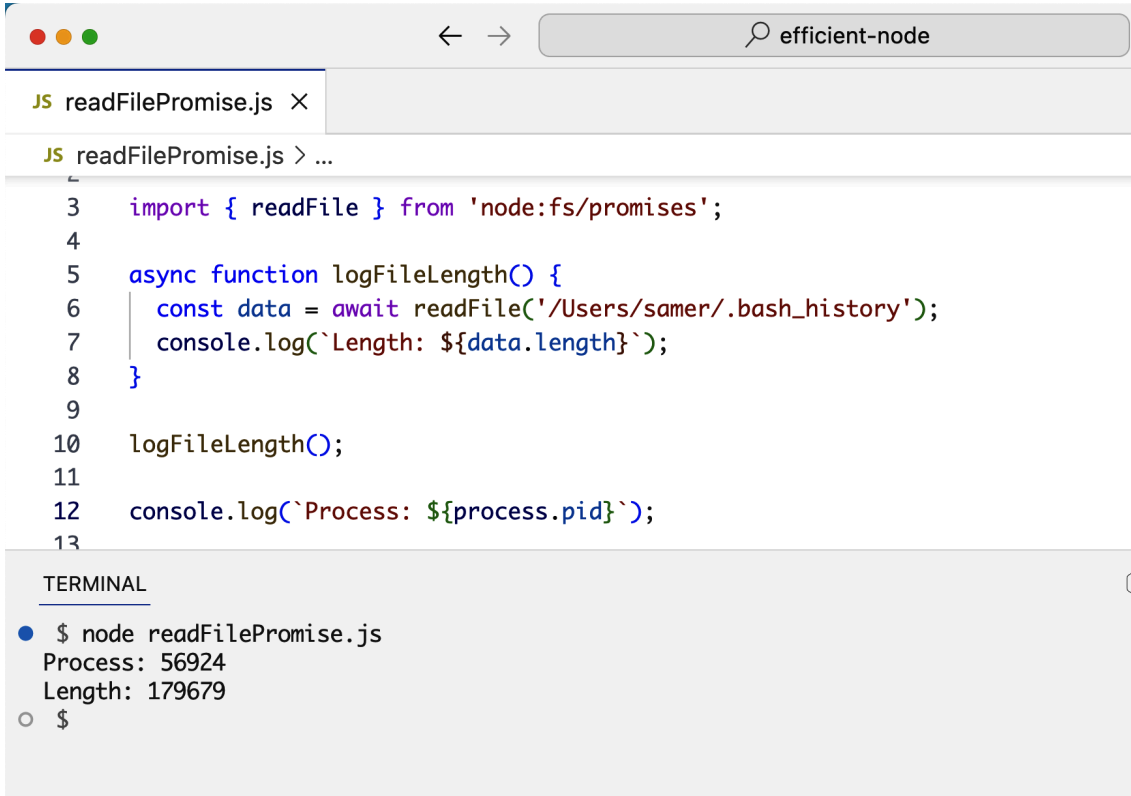
logFileLength();

console.log(`Process: ${process.pid}`);
```

Note how the `readFile` method here is imported from `node:fs/promises`. This is Node's built-in *promisified* version of the `fs` module. Executing this promise-based `readFile` method will return a `Promise` object.

To access the actual data of this operation, we use the `await` keyword within an `async` function. The `await` keyword pauses the execution of the `logFileLength` function until the promise is either resolved (success) or rejected (failure). Any function that uses the `await` keyword becomes an asynchronous function that implicitly returns a `Promise` object as well.

[Figure 1-10](#) shows the output of running this code example.



```
JS readFilePromise.js X
JS readFilePromise.js > ...
3 import { readFile } from 'node:fs/promises';
4
5 async function logFileLength() {
6   const data = await readFile('/Users/samer/.bash_history');
7   console.log(`Length: ${data.length}`);
8 }
9
10 logFileLength();
11
12 console.log(`Process: ${process.pid}`);
13
```

TERMINAL

```
● $ node readFilePromise.js
Process: 56924
Length: 179679
○ $
```

Figure 1-10. Reading a file with promises

`Promise` objects make code easier to understand and deal with. Note the similarity of the flow of code when reading a file synchronously and asynchronously with a `Promise` object. With `Promise` objects, we get to use Node's non-blocking model without needing to deal with callback functions. `Promise` objects with the `async` / `await` syntax make the code easier to read, especially when there are multiple asynchronous opera-

tions that depend on each other. With callbacks, things become a lot more complicated, whereas with promises, we just add more `await` lines.

We'll learn more about events, callbacks, promises, and the `async/await` syntax in [Chapter 3](#).

Node Built-In Modules

Armed with a simple non-blocking model, Ryan Dahl and many early contributors to Node got to work and implemented many low-level modules to offer asynchronous APIs for features like reading and writing files, sending and receiving data over a network, compressing and encrypting data, and dozens of other features.

We've already looked at simple examples of using the `node:http` and `node:fs` modules. To see the list of all built-in modules you get with Node, you can use this line (in a REPL session, as shown in [Figure 1-11](#)):

```
require('repl').builtinModules
```

```
TERMINAL node + - [ ] ...
○ $ node
>
> require("repl").builtinModules
[
  'assert',           'assert/strict',    'async_hooks',
  'buffer',           'child_process',    'cluster',
  'console',          'constants',         'crypto',
  'dgram',             'diagnostics_channel', 'dns',
  'dns/promises',      'domain',            'events',
  'fs',                'fs/promises',       'http',
  'http2',             'https',             'inspector',
  'inspector/promises', 'module',            'net',
  'os',                'path',              'path/posix',
  'path/win32',        'perf_hooks',        'process',
  'punycode',          'querystring',       'readline',
  'readline/promises', 'repl',              'stream',
  'stream/consumers',  'stream/promises',   'stream/web',
  'string_decoder',    'sys',               'timers',
  'timers/promises',  'tls',               'trace_events',
  'tty',               'url',               'util',
  'util/types',        'v8',                'vm',
  'wasi',              'worker_threads',    'zlib'
]
> |
```

Figure 1-11. Node's built-in modules

This is basically the list of things you need to learn to master Node. Well, not all of it. Depending on the version of Node, this list might include deprecated (or soon to be deprecated) modules. You also might not need many of these modules depending on the scope of your work and many other factors. For example, instead of using the native HTTPS capabilities of Node, you can simply put your Node HTTP server behind a reverse proxy like nginx or a service like Cloudflare. Similarly, you would need to learn a module like `wasi` only if you're working with WebAssembly.

Note how a few of these modules are included twice, one with a `/promises` suffix. These are the modules that support both the callback and the promise patterns.

TIP

Not all Node modules will be included in this list. Prefix-only modules and other experimental modules do not show up here. Examples include modules like `node:test`, `node:sea`, and `node:sqlite`. For a full list of modules and their development status, check out the stability overview table in the [Node documentation](#).

It's good to get familiar with this list now and get a taste of what you can do with Node. [Table 1-1](#) shows some of the important modules with a description of the main tasks you can do with them.

Table 1-1. Key Node modules

Module	Task
<code>node:assert</code>	Verify invariants for testing
<code>node:buffer</code>	Represent and handle binary data
<code>node:child_process</code>	Run shell commands and fork processes
<code>node:cluster</code>	Scale a process by distributing its load across multiple workers
<code>node:console</code>	Output debugging information
<code>node:crypto</code>	Perform cryptographic functions
<code>node:dns</code>	Perform name resolutions like IP address lookup
<code>node:events</code>	Define custom events and handlers
<code>node:fs</code>	Interact with the filesystem
<code>node:http</code>	Create HTTP servers and clients
<code>node:net</code>	Create network servers and clients
<code>node:os</code>	Interact with the operation system
<code>node:path</code>	Handle paths for files and directories
<code>node:perf_hooks</code>	Measure and analyze applications performance
<code>node:stream</code>	Handle large amounts of data efficiently
<code>node:test</code>	Create and run JavaScript tests
<code>node:timers</code>	Schedule code to be executed at a future time
<code>node:url</code>	Parse and resolve URL objects
<code>node:util</code>	Access useful utility functions
<code>node:zlib</code>	Compress and decompress data

We'll see many examples of using these modules throughout the book. Some of these modules have entire chapters focusing on them.

Node Packages

Node ships with the powerful npm package manager. We did not have a package manager in the JavaScript world before Node, so npm was nothing short of revolutionary. It changed the way we work with JavaScript.

You can build many features in a Node application by using code that's freely available on npm. The npm registry has more than a million packages that you can install and use in your Node servers. The package manager is reliable and comes with a simple CLI. The main `npm` command offers simple ways to install and maintain third-party packages, share your own code, and reuse it too.

TIP

You can install packages for Node from other package registries as well. For example, you can install them directly from GitHub.

Together, npm and Node's module systems make a big difference when you work with any JavaScript system, not just the JavaScript that you execute on backend servers or web browsers. For example, if you have a fancy fridge monitor that happens to run on JavaScript, you can use Node and npm for the tools to package, organize, and manage dependencies, and then bundle your code and ship it to your fridge!

The packages that you can run on Node come in all shapes and forms. Some are small and dedicated to specific programming tasks, some offer tools to assist in the lifecycles of an application, and others help developers every day to build and maintain big and complicated applications. Here are a few examples of some of my favorite tools available from npm:

ESLint

A tool that you can include in any Node applications and use to find problems with your JavaScript code (and, in some cases, automati-

cally fix them). You can use ESLint to enforce best practices and consistent code style, but ESLint can help point out potential runtime bugs too. You don't ship ESLint in your production environments; it's just a tool that can help you increase the quality of your code as you write it.

Prettier

An opinionated code formatting tool. With Prettier, you don't have to manually indent your code, break long code into multiple lines, remember to use a consistent style for the code (for example, always use single or double quotes, always use semicolons, never use semicolons, etc.). Prettier automatically takes care of all of that.

Webpack

A tool that assists with asset bundling. The Webpack Node package makes it very easy to bundle your multiframe frontend application into a single file for production, and to compile JavaScript extensions (like JSX for React) during that process. This is an example of a Node tool that you can use on its own. You do not need a Node web server to work with Webpack.

TypeScript

A tool that adds static typing and other features to the JavaScript language. It is useful because it can help developers catch errors before the code is run, making it easier to maintain and scale large codebases. TypeScript's static typing can also improve developer productivity by providing better code autocompletion and documentation in development tools.

All of these tools (and many more) enrich the experience of creating and maintaining JavaScript applications, both on the frontend and the backend. Even if you choose not to host your frontend applications on Node, you can still use Node for its tools. For example, you can host your frontend application with another framework such as Ruby on Rails and use Node to build assets for the Rails asset pipeline.

We will learn more about these tools (and others) in [Chapter 10](#).

Arguments Against Node

Node's approach to handling code in an asynchronous and non-blocking manner is a unique model of thinking and reasoning about code. If you've never done it before, it will feel weird at first. You need time to get your head wrapped around this model and get used to it.

Node's module system was originally built around CommonJS, which has since been largely replaced by the newer ES modules standard in JavaScript. While Node supports both systems, using them together can be confusing, especially for beginners. The differences in how CommonJS and ES modules handle imports and exports can lead to inconsistent code and compatibility issues.

Node developers rely on many third-party libraries and dependencies, and npm stores them all in one large *node_modules* folder, which can become bloated and difficult to manage. It's not uncommon for a Node project to use hundreds of third-party packages, which require management and oversight. As packages are regularly updated or abandoned, it becomes necessary to closely monitor and update all packages used within a project, resolving any version conflicts, replacing deprecated options, and ensuring that your code is not vulnerable to any of the security problems these packages might introduce.

Security in general is one of the strongest arguments against Node. A Node script has unrestricted access to the filesystem, network, and other system resources. This can be dangerous when running third-party code because malicious scripts could exploit these permissions. Node is introducing a new permission model to restrict access to specific resources during execution. You can restrict a Node process from accessing the filesystem, spawning new processes, using worker threads, using native add-ons, and using WebAssembly. However, these restrictions are not enabled by default.

Another limitation in Node is the lack of built-in tools for tasks like validating types, linting, and formatting code. Developers typically have to rely on third-party packages to add these features. While there are plenty

of great options, setting up and configuring them can be time-consuming and adds extra steps before you can start coding.

Additionally, Node is optimized for I/O operations and high-level programming tasks, but it may not be the best choice for CPU-bound tasks, such as image and video processing, which require a lot of computational power. Because Node is single-threaded, meaning that it can use only one core of a CPU at a time, performing tasks that require a lot of CPU processing power might lead to performance bottlenecks. JavaScript itself is not the best language for high-performance computation, as it is less performant than languages like C++ or Rust.

Finally, JavaScript, the language you use in Node, has one important argument against it. It is a dynamically typed language, which means objects don't have explicitly declared types at compile time, and they can change during runtime. This is fine for small projects, but for bigger ones, the lack of strong typing can lead to errors that are difficult to detect and debug, and it generally makes the code harder to reason with and maintain.

Summary

Node is a powerful framework for building backend services. It wraps the V8 JavaScript engine to enable developers to execute JavaScript code in a simple way, and it is built on top of a simple, event-driven, non-blocking model that makes it easy for developers to create efficient and scalable applications.

In Node, asynchronous operations are handled with callback functions or **Promise** objects. Callbacks and promises are simple implementations of a one-time event that gets handled with one function. Promises are a better alternative to callbacks as they offer a more readable syntax and can be structured in a way to allow for more control over the code.

The built-in modules in Node provide a low-level framework on which developers can base their applications so that they don't start from scratch. Node's module system allows developers to organize their code into reusable modules that can be imported and used in other parts of the

application. Node has a large and active community that has created many popular packages that can be easily integrated into Node projects. These packages can be found and downloaded from the npm registry.

In the next chapter, we'll explore Node's CLI and REPL mode and learn how Node loads and executes modules.